

WEB DEV-II Detailed Answer Book

Engineering Level Explanations with Detailed Code Examples

Q1. Explain Event Handling in JavaScript using addEventListener() with different event types.

Event handling is one of the most important concepts in JavaScript. It allows webpages to respond to user actions such as clicking buttons, typing text, moving the mouse, or submitting forms. JavaScript uses events to make webpages interactive and dynamic.

The addEventListener() method is used to attach an event handler to an HTML element. It allows multiple events to be attached to the same element without overwriting previous events.

Syntax:

```
element.addEventListener(event, function, useCapture);
```

Common Event Types:

1. click – triggered when user clicks.
2. mouseover – triggered when mouse moves over element.
3. keydown – triggered when keyboard key is pressed.
4. submit – triggered when form is submitted.
5. change – triggered when input value changes.

Advantages of addEventListener():

- Multiple event listeners can be added.
- Better code organization.
- Supports event bubbling and capturing.
- Can remove events using removeEventListener().

```
<!DOCTYPE html>
<html>
<head>
<title>Event Example</title>
</head>
<body>

<button id="btn">Click Me</button>
<script>
```

```

const button = document.getElementById("btn");

button.addEventListener("click", function(){
    alert("Button Clicked");
});

button.addEventListener("mouseover", function(){
    console.log("Mouse over button");
});
</script>

</body>
</html>

```

Q2. Explain Event Propagation. Differentiate between Bubbling and Capturing.

Event propagation describes how events travel in the DOM hierarchy. When an event occurs on an element, the event moves through different phases in the DOM tree.

Phases of Event Propagation:

1. Capturing Phase – Event moves from root element to target element.
2. Target Phase – Event reaches the target element.
3. Bubbling Phase – Event moves back from target to parent elements.

Event Bubbling:

In bubbling, the event starts from the target element and moves upward toward parent elements.

Event Capturing:

In capturing, the event starts from the top ancestor and moves downward to the target element.

Difference:

- Bubbling moves upward.
- Capturing moves downward.
- Bubbling is default behavior.
- Capturing requires third parameter true.

Event propagation is useful in event delegation and handling complex UI interactions.

```

<div id="parent">
  <button id="child">Click</button>
</div>
<script>

```

```

document.getElementById("parent")
.addEventListner("click", function(){
    console.log("Parent Clicked");
}, true);

document.getElementById("child")
.addEventListner("click", function(){
    console.log("Child Clicked");
});
</script>

```

Q3. Explain Execution Context in JavaScript including GEC, FEC and Call Stack.

Execution Context is the environment where JavaScript code is executed. JavaScript engine creates execution contexts to manage variables, functions, and execution flow.

Types of Execution Context:

1. Global Execution Context (GEC): Created when program starts. It creates global object and this keyword.
2. Function Execution Context (FEC): Created whenever a function is invoked. Each function gets its own local memory.

Call Stack:

Call stack is a stack data structure that manages execution contexts. When a function is called, its context is pushed into the stack. After execution, it is removed from the stack.

Working:

- Global context enters first.
- Function calls create FECs.
- Stack follows Last In First Out (LIFO).

Execution context is essential for understanding scope, hoisting, and asynchronous behavior.

```

function first(){
    console.log("First Function");
    second();
}

function second(){
    console.log("Second Function");
}

first();
/*

```

```
Call Stack Flow:
1. Global Context
2. first()
3. second()
*/
```

Q4. Explain the lifecycle of a Promise in JavaScript.

Promises are used to handle asynchronous operations in JavaScript. They provide a cleaner alternative to Callback Hell and improve readability.

Callback Hell:

Callback Hell occurs when callbacks are nested deeply, making code difficult to maintain.

Promise States:

1. Pending – Initial state.
2. Fulfilled – Operation successful.
3. Rejected – Operation failed.

Promise Consumers:

- .then() – handles success.
- .catch() – handles errors.
- .finally() – executes always.

Advantages:

- Better readability.
- Better error handling.
- Easier chaining.

Promises are widely used in APIs and asynchronous JavaScript programming.

```
const myPromise = new Promise((resolve, reject) => {

    let success = true;

    if(success){
        resolve("Operation Successful");
    } else {
        reject("Operation Failed");
    }
});
myPromise
```

```
.then((result) => {
    console.log(result);
})
.catch((error) => {
    console.log(error);
})
.finally(() => {
    console.log("Promise Completed");
});
```

Q5. Explain async/await with error handling using try...catch.

async and await are modern JavaScript features used for asynchronous programming. They simplify Promise handling and make code easier to read.

async:

An async function always returns a Promise.

await:

The await keyword pauses execution until the Promise resolves.

Error Handling:

Errors are handled using try...catch blocks.

Advantages:

- Cleaner syntax.
- Easier debugging.
- Better readability.
- Sequential asynchronous execution.

async/await is commonly used with APIs and database operations.

```
async function fetchData(){
    try{
        let response = await fetch(
            "https://jsonplaceholder.typicode.com/users");

        let data = await response.json();

        console.log(data);
    } catch(error){
        console.log("Error:", error);
    }
}
```

```
    }  
  }  
  
  fetchData();
```

Q6. Explain React Component Lifecycle using useEffect Hook.

Every React component goes through different lifecycle phases. These phases are mounting, updating, and unmounting.

Mounting:

Component is created and inserted into DOM.

Updating:

Occurs when state or props change.

Unmounting:

Component is removed from DOM.

In functional components, useEffect hook is used to manage lifecycle behavior.

useEffect Syntax:

```
useEffect(() => { }, [dependencies]);
```

Dependency Array:

- Empty array → runs once.
- With dependencies → runs on changes.
- Without array → runs every render.

useEffect is widely used for API calls, timers, subscriptions, and side effects.

```
import React, { useEffect } from 'react';  
  
function App(){  
  useEffect(() => {  
    console.log("Component Mounted");  
  
    return () => {  
      console.log("Component Unmounted");  
    };  
  });  
}
```

```
    }, []);

    return(
      <h1>Hello React</h1>
    );
  }
}

export default App;
```

Q9. React Component with Two Buttons, Counter State, and Popup

React provides hooks like `useState` and `useEffect` to manage state and side effects in functional components. In this program, one button displays a popup every time it is clicked, while the second button only updates the

`useState` is used to store the counter value and popup trigger value. `useEffect` is used to control the popup, so

Code Example:

```
import React, { useState, useEffect } from "react";

function PopupCounter() {
  const [count, setCount] = useState(0);
  const [popupClick, setPopupClick] = useState(0);

  useEffect(() => {
    if (popupClick > 0) {
      alert("Popup button clicked!");
    }
  }, [popupClick]);

  return (
    <div>
      <h2>Counter: {count}</h2>

      <button onClick={() => setPopupClick(popupClick + 1)}>
        Show Popup
      </button>

      <button onClick={() => setCount(count + 1)}>
        Increase Counter
      </button>
    </div>
  );
}

export default PopupCounter;
```

Thus, `useState` manages the counter and `useEffect` controls when the popup appears.

Q10. User Information Form in React

A form in React is used to collect input from the user. React forms are controlled components where form values

The form contains Name, Age, Gender, and Language fields. On submission, the entered data is displayed on the s

Code Example:

```
import React, { useState } from "react";

function UserForm() {
  const [user, setUser] = useState({
    name: "",
    age: "",
    gender: "",
    language: []
  });

  const [submittedData, setSubmittedData] = useState(null);

  const handleChange = (e) => {
    const { name, value, checked } = e.target;

    if (name === "language") {
      if (checked) {
        setUser({
          ...user,
          language: [...user.language, value]
        });
      } else {
        setUser({
          ...user,
          language: user.language.filter((lang) => lang !== value)
        });
      }
    } else {
      setUser({
        ...user,
        [name]: value
      });
    }
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmittedData(user);
  };

  return (
    <div>
      <h2>User Information Form</h2>
    </div>
  );
}

export default UserForm;
```

This program demonstrates how React handles forms using state.

Q11. Counter Application using useState

useState is a React Hook used to add state in functional components.
This counter application contains increment, decrement, and reset buttons.

Code Example:

```
import React, { useState } from "react";

function CounterApp() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Counter Application</h2>

      <h3>Count: {count}</h3>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}
```

```
export default CounterApp;
```

This application shows how useState manages dynamic data in React.

Q12. Context API and Props Drilling with Implementation

Props drilling means passing data through multiple components using props, even when intermediate components do not need the data.

Context API is used to share data globally between components without passing props manually.

Code Example:

```
import React, { createContext, useContext } from "react";
```

```
const UserContext = createContext();
```

```
function App() {  
  const user = "Priyanshu";  
  
  return (  
    <UserContext.Provider value={user}>  
      <Parent />  
    </UserContext.Provider>  
  );  
}
```

```
function Parent() {  
  return <Child />;  
}
```

```
function Child() {  
  return <GrandChild />;  
}
```

```
function GrandChild() {  
  const user = useContext(UserContext);  
  
  return <h2>Welcome, {user}</h2>;  
}
```

```
export default App;
```

Context API simplifies data sharing across nested components.

Q13. Disadvantages of Props Drilling and How Context API Solves Them

Props drilling causes unnecessary passing of data through components.

Disadvantages:

1. Code becomes lengthy.
2. Difficult maintenance.
3. Reduced readability.
4. Intermediate components receive unnecessary props.

Context API solves this by allowing direct access to data using useContext.

Code Example:

```
import React, { createContext, useContext } from "react";

const UserContext = createContext();

function App() {
  return (
    <UserContext.Provider value="Priyanshu">
      <Parent />
    </UserContext.Provider>
  );
}

function Parent() {
  return <Child />;
}

function Child() {
  return <GrandChild />;
}

function GrandChild() {
  const user = useContext(UserContext);
  return <h2>Hello, {user}</h2>;
}

export default App;
```

Thus, Context API improves readability and reduces complexity.

Q14. Routing in React with Example and Multiple Pages

Routing in React is used to navigate between multiple pages without reloading the webpage.

React Router provides BrowserRouter, Routes, Route, and NavLink components.

Installation:

```
npm install react-router-dom
```

Code Example:

```
import React from "react";
import { BrowserRouter, Routes, Route, NavLink } from "react-router-dom";

function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}

function Contact() {
  return <h2>Contact Page</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/">Home</NavLink> |
        <NavLink to="/about">About</NavLink> |
        <NavLink to="/contact">Contact</NavLink>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Routing improves navigation and user experience in React applications.

Additional Questions

Q7. Write the code - How to Fetch API in React

In React, API data is fetched using the `fetch()` method.

The `useEffect` hook is used to call the API when the component loads, and `useState` is used to store the fetched

The following program fetches data from:

<https://jsonplaceholder.typicode.com/users>

and displays the username and email of all users.

React Code:

```
import React, { useEffect, useState } from "react";

function FetchUsers() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => {
        setUsers(data);
      })
      .catch((error) => {
        console.log("Error:", error);
      });
  }, []);

  return (
    <div>
      <h2>User Details</h2>

      {users.map((user) => (
        <div key={user.id}>
          <h3>Username: {user.username}</h3>
          <p>Email: {user.email}</p>
          <hr />
        </div>
      ))}

    </div>
  );
}

export default FetchUsers;
```

Sample JSON:

```
{
  "username": "Bret",
  "email": "Sincere@april.biz"
}
```

Output:

Username: Bret
Email: Sincere@april.biz

Username: Antonette
Email: Shanna@melissa.tv

Thus, React uses `fetch()`, `useEffect`, and `useState` together to retrieve and display API data dynamically.

Q8. Create a simple webpage using HTML, CSS, and JavaScript that demonstrates button click events and event removal.

This webpage demonstrates how to use `addEventListener()` and `removeEventListener()` in JavaScript.

Features:

- A button is added on the webpage.
- A heading contains default text.
- When the button is clicked:
 - The text changes.
 - The text color changes.
- After one click, the event listener is removed using `removeEventListener()`.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Event Removal Example</title>

  <style>
    body {
      font-family: Arial;
      text-align: center;
      margin-top: 100px;
    }

    h1 {
      color: black;
    }

    button {
      padding: 10px 20px;
      font-size: 18px;
      cursor: pointer;
    }
  </style>
</head>

<body>

  <h1 id="message">Welcome to JavaScript Events</h1>

  <button id="btn">Click Me</button>

  <script>

    const button = document.getElementById("btn");
    const heading = document.getElementById("message");

    function changeText() {

      heading.innerHTML = "Button Clicked Successfully!";
      heading.style.color = "blue";

      button.removeEventListener("click", changeText);

      alert("Event Listener Removed");
    }

    button.addEventListener("click", changeText);

  </script>

</body>
</html>
```

Explanation:

1. `addEventListener()` attaches a click event to the button.
2. When the button is clicked:
 - The heading text changes.
 - The heading color changes to blue.
3. `removeEventListener()` removes the click event after the first execution.
4. After the first click, clicking the button again will not perform any action.

Thus, this example demonstrates event handling and event removal in JavaScript.